AI Final

Correct Code

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 1:

```
import math

def calculate_opposite_side():
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.
```

```
    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

```
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_3979/3696660382.py in <cell line: 0>()
     20 angle = 30  # degrees
     21 hyp = 10    # units
---> 22 opposite = calculate_opposite_side(angle, hyp)
     23 print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")

TypeError: calculate_opposite_side() takes 0 positional arguments but 2 were given
```

Reflection: The function has no parameters. Python doesn't know what angle_degrees is. The number of inputs in *def* should match how the function is called.

Correct Code:

```
import math

# Add 'angle_degrees' and 'hypotenuse' inside the parentheses here:
def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
```

```
angle = 30  # degrees
hyp = 10    # units

# Now the function is ready to accept these 2 arguments
opposite = calculate_opposite_side(angle, hyp)

print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 2:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = "30"
hyp = "5"
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

```
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_3979/388345200.py in <cell line: 0>()
     20 angle = "30"
     21 hyp = "5"
---> 22 opposite = calculate_opposite_side(angle, hyp)
```

```
    23 print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")

/tmp/ipykernel_3979/388345200.py in calculate_opposite_side(angle_degrees, hypotenuse)
    13         float: The length of the opposite side.
    14     """
---> 15     angle_radians = math.radians(angle_degrees)
    16     opposite_side = hypotenuse * math.sin(angle_radians)
    17     return opposite_side

TypeError: must be real number, not str
```

Reflection: Based on what the code error said, the bug comes from a string text. The code was expecting a number. This helps me understand what string text is, why it is important, and when it is important to use versus not use.

Correct Code:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    # Convert inputs to float inside the function just to be safe
    angle_radians = math.radians(float(angle_degrees))
    opposite_side = float(hypotenuse) * math.sin(angle_radians)
    return opposite_side

# Example usage:
# Option 1: Remove the quotes (Best practice)
angle = 30
hyp = 5

# Option 2: If they MUST stay as strings, the function now handles them with float()
opposite = calculate_opposite_side(angle, hyp)

print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 3:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
```

```
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse + math.sin(angle_radians)
    hypotenuse = float(hypotenuse)
    angle_degrees = float(angle_degrees)
    return opposite_side

# Example usage:
angle = 30
hyp = 10
opposite_side = hypotenuse + math.sin(angle_radians)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

```
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3979/83175123.py in <cell line: 0>()
     22 angle = 30
     23 hyp = 10
---> 24 opposite_side = hypotenuse + math.sin(angle_radians)
     25 print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")

NameError: name 'hypotenuse' is not defined
```

Reflection: I think the obvious problem was a definition problem. On line 22 and line 23, I defined a variable called hyp. Because I didn't define what hyp meant and did not clarify that it meant hypotenuse, Python didn't know that these are the same thing. This helps me better understand coding, because you must be specific about definitions.

Correct Code:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
```

```
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.
    """
    # 1. Convert degrees to radians
    angle_radians = math.radians(float(angle_degrees))

    # 2. Use multiplication (*), not addition (+)
    # Formula: opposite = hypotenuse * sin(angle)
    opposite_side = float(hypotenuse) * math.sin(angle_radians)

    return opposite_side

# Example usage:
angle = 30
hyp = 10

# 3. Call the function and assign the result to 'opposite'
opposite = calculate_opposite_side(angle, hyp)

# 4. Print the result
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 4:

```
import math

def calculate_triangle(angle_degrees, hypotenuse):
    if hypotenuse <= 0:
        raise ValueError("Hypotenuse must be positive")

    if not (0 < angle_degrees < 90):
        raise ValueError("Angle must be between 0 and 90 degrees")

    angle_radians = math.radians(angle_degrees)

    opposite = hypotenuse * math.sin(angle_radians)
    adjacent = hypotenuse * math.cos(angle_radians)

    return opposite, adjacent
```

```
# Example usage:
angle = 9332 # degrees
hyp = 60   # units
opposite = calculate_opposite_side(angle, hyp)
print(int(opposite))
```

0

Reflection: This one was really interesting because I knew the only change I made was to the print line and it completely changed the output. I deleted the information that the code originally had and saw what would happen. I think something else that added to the output being 0 was my angle input which was 9332º. What I found interesting was that my code didn't raise an error, but instead returned a value. Some things that are issues in this line of code are:

- **Using int() hides real values**
- **Angle outside valid range**
- **Calling wrong function name**
- **Old code still running**

Correct Code:

```
import math

def calculate_triangle(angle_degrees, hypotenuse):
    # If you want to support angles larger than 90,
    # we should use the modulo operator (%) to keep them within 0-360 range
    normalized_angle = angle_degrees % 360

    angle_radians = math.radians(normalized_angle)

    opposite = hypotenuse * math.sin(angle_radians)
    adjacent = hypotenuse * math.cos(angle_radians)

    return opposite, adjacent

# Example usage:
angle = 9332  # This is actually equivalent to 332 degrees
hyp = 60

# 1. FIX: Call the correct function name
# 2. FIX: Unpack BOTH return values (opposite and adjacent)
opposite, adjacent = calculate_triangle(angle, hyp)
```

```
# 3. FIX: Use float formatting instead of int()
print(f"Opposite: {opposite:.2f}")
```

Bugged Code 5:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side


# Example usage:
angle = 8 # degrees
hyp = 10   # units
opposite, adjacent = calculate_triangle(angle, hyp)
opposite, adjacent = calculate_triangle(angle + 10, hyp)
print(opposite)  # user thinks it's first result
```

3.090169943749474

Reflection: This code was interesting because it didn't necessarily give me an error. There are several reasons for the odd answer and why it is a state overwrite bug including: two results were computed, but both were stored in the same variable causing the first to be lost.

Correct Code:

```
import math
```

```
# Your defined function
def calculate_opposite_side(angle_degrees, hypotenuse):
    angle_radians = math.radians(angle_degrees)
    # We'll return BOTH opposite and adjacent to match your usage below
    opposite_side = hypotenuse * math.sin(angle_radians)
    adjacent_side = hypotenuse * math.cos(angle_radians)
    return opposite_side, adjacent_side

# Example usage:
angle = 8
hyp = 10

# 1. Use unique names for the first calculation
opp1, adj1 = calculate_opposite_side(angle, hyp)

# 2. Use unique names for the second calculation (angle + 10 = 18 degrees)
opp2, adj2 = calculate_opposite_side(angle + 10, hyp)

# 3. Now you can print exactly which one you want
print(f"First result (8°): {opp1:.2f}")
print(f"Second result (18°): {opp2:.2f}")
```

Bugged Code 6:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):

"""
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

"""

# Example usage:
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

```
File "/tmp/ipykernel_12705/3027683670.py", line 5
    """
    ^
IndentationError: expected an indented block after function definition on line 3
```

Reflection: This code output confused me especially because it spit out a url. Even though this output was unique, I think the problem was a pretty basic one which was just an indentation problem. This helps me understand how to input code a little better because the issue was from beginning to define a function but not putting anything inside the definition.

Correct Code: This is what correction would be successful

```
def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side...
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side
```

Bugged Code 7:

```
import math

angle_radians = math.radians(angle_degrees)
opposite_side = hypotenuse * math.sin(angle_radians)
return opposite_side

# Example usage:
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is: {opposite:.2f}")
```

```
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_12705/2721410720.py in <cell line: 0>()
      1 import math
      2
----> 3 angle_radians = math.radians(angle_degrees)
      4 opposite_side = hypotenuse * math.sin(angle_radians)
```

```
5 return opposite_side
```

NameError: name 'angle_degrees' is not defined

Reflection: Here, I messed around with deleting certain inputs and leaving out information like definitions to see how the code would react. I think it output the error because I use a return statement on Line 5. In Python, return can only exist inside a function (a block starting with def). Because I don't have a def line, Python is confused.

Correct Code:

```
import math

# 1. Define the function properly
def calculate_opposite_side(angle_degrees, hypotenuse):
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# 2. Set your variables
angle = 30  # degrees
hyp = 10    # units

# 3. Call the function
opposite = calculate_opposite_side(angle, hyp)

# 4. Print the result
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 8:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):

        Calculates the length of the opposite side of a right triangle
        given an angle in degrees and the hypotenuse.


    Args:
            angle_degrees (float): The angle of the triangle in degrees.
```

```
        hypotenuse (float): The length of the hypotenuse.


    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
                opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
            angle = 30  # degrees
hyp = 10    # units
                opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")

 File "<tokenize>", line 9
    Args:
    ^
IndentationError: unindent does not match any outer indentation level
```

Reflection: I think what went wrong here was a string issue. The string was not open on line 9. Python thinks the word "Calculates" is a variable name and the word "Args:" is a command. Because of this it tries to read those sentences as code, gets confused by the indentation. I think there is also a problem with indentions.

Correct Code:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
```

```
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage (no indentation here because this is the main script)
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)

print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Bugged Code 9:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = 30  # degrees
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
result = calculate_opposite_side(30, 10)

if result == 5.0:
    print("Success!")
```

```
else:
    print("Failure!") # This can sometimes trigger due to 5.0000000000000001
```

For an angle of 30 degrees and hypotenuse of 10, the opposite side is: 5.00
Failure!

Reflection: Here, I played around with adding a variable line of code like an "if this, then that" to see how my code would respond. The output was interesting because it typed 'Failure' when I thought it would type 'Success'. I think the reasoning has to do with the line of code

> **'print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is: {opposite:.2f}")'**

So, the program produced "Failure" because floating-point arithmetic created a rounding error. Although the result appeared to be 5.00 when printed, its internal representation differed from 5.0, causing the equality check to fail.

Correct Code:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = 30
hyp = 10
result = calculate_opposite_side(angle, hyp)

# FIX: Use math.isclose to handle tiny rounding errors
if math.isclose(result, 5.0, rel_tol=1e-9):
    print(f"Result: {result}")
    print("Success!")
else:
    print(f"Result: {result}")
    print("Failure!")
```

Bugged Code 10:

```
import math

def calculate_opposite_side(angle_radians, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.

    Args:
        angle_degrees (float): The angle of the triangle in degrees.
        hypotenuse (float): The length of the hypotenuse.

    Returns:
        float: The length of the opposite side.
    """
    angle_radians = math.radians(angle_degrees)
    opposite_side = hypotenuse * math.sin(angle_radians)
    return opposite_side

# Example usage:
angle = 30  # radians
hyp = 10    # units
opposite = calculate_opposite_side(angle, hyp)
print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

```
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_12705/3879166060.py in <cell line: 0>()
     20 angle = 30  # radians
     21 hyp = 10    # units
---> 22 opposite = calculate_opposite_side(angle, hyp)
     23 print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
     24

/tmp/ipykernel_12705/3879166060.py in calculate_opposite_side(angle_radians, hypotenuse)
     13         float: The length of the opposite side.
     14     """
---> 15     angle_radians = math.radians(angle_degrees)
     16     opposite_side = hypotenuse * math.sin(angle_radians)
     17     return opposite_side
```

NameError: name 'angle_degrees' is not defined

Reflection: Here, I purposefully messed with the units of the inputs trying to confuse the code. The output gives me evidence that the issue was caused because the function parameter name (angle_radians) does not match the variable used inside the function (angle_degrees).

Correct Code:

```
import math

def calculate_opposite_side(angle_degrees, hypotenuse):
    """
    Calculates the length of the opposite side of a right triangle
    given an angle in degrees and the hypotenuse.
    """
    # Convert degrees to radians because math.sin() expects radians
    angle_radians = math.radians(angle_degrees)

    # Using the formula: opposite = hypotenuse * sin(theta)
    opposite_side = hypotenuse * math.sin(angle_radians)

    return opposite_side

# Example usage:
angle = 30  # In degrees
hyp = 10    # In units
opposite = calculate_opposite_side(angle, hyp)

print(f"For an angle of {angle} degrees and hypotenuse of {hyp}, the opposite side is:
{opposite:.2f}")
```

Report:

Based on these purposeful code mistakes and bugs, I've learned a lot about code and why certain things are important.

For example, it is important to be very specific when coding. Computers are literal. Unlike a human who knows hyp is short for hypotenuse, Python treats them as two different

things. What I have learned from this is that every variable must be defined before it is used. I also learned that if you name a variable inside a function, it cannot be read by the rest of the program. Likewise, when having an LLM assist me in my code, it is important to ensure your variable names are consistent across the entire snippet, or the AI might assume you are referring to two different things.

Another thing I learned is the difference between strings and integers. For example “30” is not the same as 30 in a line of code. Mathematical functions like math.sin() require "Real Numbers", and so it is important to always ensure inputs are converted to the correct type before performing calculations.

Similarly to the importance of being specific, it is important to be consistent with units because your code will not assume what is correct. That being said, I have learned that sometimes including a "translation" step to bridge the gap between human intuition and computer requirements can be helpful.

Additionally, I have learned about the importance of indents, structure, and hierarchy in code. For example, often my code would run smoothly if I were to indent with “”’”, but the second I added a couple more quotation marks, my code would run into an error. Something that my LLM said that made sense was that “An IndentationError or a missing docstring quote (""") isn't just a typo; it breaks the "map" the computer uses to read your instructions.”

This was an interesting activity because it forced me to reverse engineer code in a way which helped me understand it better.